

Certified but Compromised: Breaking and Fixing Lightweight Proofs of Inference

Final Report — Trustworthy Machine Learning, Spring 2026

Ofek Lavi
Tel Aviv University
Tel Aviv, Israel
ofeklavi@mail.tau.ac.il

Erel Brazilay
Tel Aviv University
Tel Aviv, Israel
erelbrazilay@mail.tau.ac.il

Edo Kadosh
Tel Aviv University
Tel Aviv, Israel
edo@mail.tau.ac.il

Abstract

When a large model runs in the cloud, its users cannot tell whether an answer really came from the agreed weights. zkSNARK proofs of inference solve this but take minutes per query, so Anchuri et al. (SaTML 2026) proposed a lightweight alternative: the server commits to the whole computation and the client re-checks one random path through the network. We reimplement this protocol and break it by changing a single neuron: the verifier still accepts the tampered answer 99.8% of the time, a backdoor built this way costs nothing in clean accuracy, and on a real OPT-6.7B one neuron changes the next token of every prompt we tried. We show that smarter sampling rules do not fix this. We then design a new defence that checks *every* layer at once. It verifies one random linear combination per layer (Freivalds’ check) against a Reed–Solomon/Merkle commitment to the weights, in exact integer arithmetic. It catches any wrong value with probability $1 - 2^{-\lambda}$ and needs no trusted setup. On models from LeNet-5 to Llama-2-13B it rejected all 2,895 attacks and accepted all 8,709 honest queries. Its prover is one to four orders of magnitude faster than published zkSNARK provers on the same models (as reported on their own hardware), and about 50× faster than zkLLM’s code run on our GPU. The price is a larger proof and a slower verifier.

Keywords

verifiable inference, proof of inference, integrity attacks, Freivalds’ algorithm, zkML

1 Introduction

Suppose a company pays a cloud provider to answer its customers with a specific large language model, for example a model it has licensed or fine-tuned. Every answer that comes back is just text. Nothing in it shows that the provider really ran that model with the agreed weights. It could have used a cheaper or more heavily quantised copy, a model with a few weights edited, or a model that behaves normally except on certain inputs. The customer cannot simply re-run the model to check: not having to run it is the whole reason to use the cloud. A *proof of inference* closes this gap. With every answer the provider also sends a piece of evidence, and the client runs a cheap check that passes only if the answer came from the agreed weights on this input.

The strongest proofs of inference are zkSNARKs [6, 12, 20, 24, 26]. Their proofs are tiny and quick to check, but producing one takes far longer than the inference itself. zkLLM, for example, needs about ten minutes on an A100 GPU for one Llama-2-7B prompt [26]. Anchuri et al. [2] recently proposed a much lighter

approach. The provider commits to the full execution trace (every activation) with a Merkle tree. The client then checks the computation along *one random path* from an output neuron back to the input, against a commitment to the weights. Proving drops from minutes to milliseconds. The security argument is statistical: a different model produces a trace that differs from the correct one in many places, so a random path is very likely to hit one of them.

What we did. Our intermediate report started from one observation. Such a scheme certifies that the served model is *functionally close* to the committed one, not that it *is* the committed one, and backdoored models [15] are the natural counterexample. We reimplemented the protocol from scratch (no code was released) and reproduced its main claims. Then we asked what happens if the adversary does not swap the model but changes only what it needs to. The answer is simple and severe. A prover can run the right model, overwrite *a single neuron*, and honestly recompute everything after it (Figures 1a and 1b). The resulting trace is wrong in exactly one place. A random path passes through that place with probability $1/N$ in a layer of width N , so on our MNIST network the verifier accepts the tampered answer 99.8% of the time. Triggering the change only on chosen inputs gives a backdoor with no accuracy cost at all on clean inputs, and on a real OPT-6.7B one neuron changes the next token of all 40 prompts we tried. The paper notes that one path cannot detect a single changed node with probability above $1/N$. It argues that this does not appear to be a practical limitation for the substitute-model adversary it targets, and suggests sampling more paths, or sampling adaptively, as mitigations. We show that the attack is practical, and that neither mitigation repairs it cheaply.

This led us to a different design (Figure 1c). Instead of checking a few neurons exactly, we check *every* layer at once with one random linear combination of all its neurons (Freivalds’ algorithm [11]). When the client does not hold the weights, it checks this combination against a Reed–Solomon-encoded, Merkle-committed copy of them, in the style of Ligerio [1]. The client therefore still stores only 32 bytes per weight matrix. To make the check exact, the model runs as an integer (int8) network that gives bit-for-bit the same result on the provider’s GPU and the client’s CPU. The prover’s extra work is a fixed cost per query that does not grow with the input. The chance of catching a wrong value, anywhere in the network, rises from $1/N$ to $1 - 2^{-\lambda}$.

Contributions.

- **An attack** on the protocol of [2]. Changing one neuron changes the answer. The verifier accepts the result with

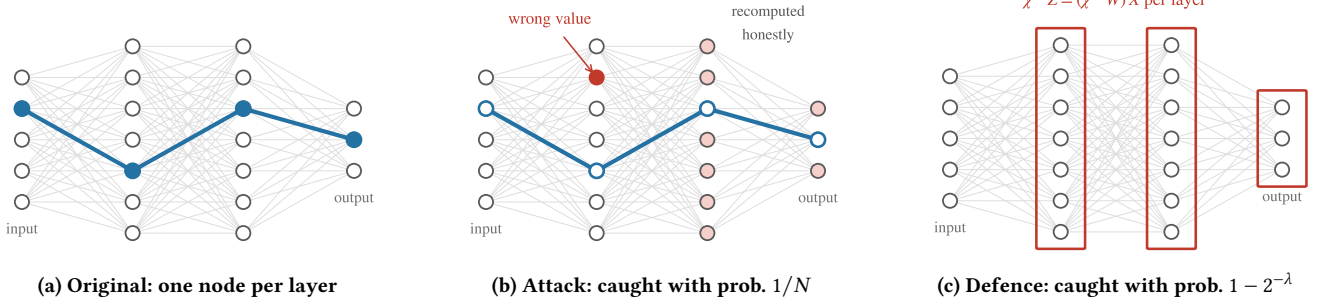


Figure 1: Checking one path versus checking every layer.

probability $1 - 1/N$ (99.6–99.8% on our MNIST model), including a trigger backdoor with zero clean-accuracy gap and 100% attack success. On the real OPT-6.7B it changes the next token of 40 of 40 prompts.

- **An impossibility argument** for sampling-based fixes. Uniform sampling is already the best rule that ignores the claimed values. The natural rule that looks at them has a blind spot, and our attacker reaches exactly zero detection through it. Reaching 2^{-40} forces the original protocol to open essentially the whole model and trace.
- **A new defence** with soundness error $2^{-\lambda}$ and no trusted setup. It covers whole CNNs (convolutions, pooling, residual blocks) and transformers (normalisation, GELU/SiLU, RoPE, softmax attention), with the weights either committed or known to the client.
- **A same-model evaluation** against the original paper and 17 other systems: 6 trained image classifiers and 10 LLM architectures up to 13B parameters, all measured at full depth on one GPU. Every attack was rejected. The prover is one to four orders of magnitude faster than the zkSNARKs as reported on their own hardware, and 48–56× faster than zkLLM’s code on the same GPU. With real OPT weights, the int8 models stay usable (perplexity within 7–33% of fp32). All code and raw measurements are released with this report.

2 Related Work

zkSNARK proofs of inference. A long line of work compiles neural networks into succinct zero-knowledge proofs. For CNNs these include sum-check/GKR provers [14] such as zkCNN [20] and zkPyTorch [30], and pairing- or Plonk-based systems such as vCNN [18], ZEN [8], ZENO [9], ZKML [6], EZKL [33] and Bionetta [32]. Mystique [29] is an interactive, VOLE-based alternative. For LLMs they include zkLLM [26], zkGPT [24], Deep-Prove [12], ZKTorch [5] and Jolt Atlas [4]. These systems give computational soundness and small proofs, but their provers are far slower than inference: from half a second for LeNet-5 to minutes or hours per query for larger models.

When the verifier knows the weights. SafetyNets [13] uses interactive proofs for networks with quadratic activations. Slalom [28] checks every linear layer of a GPU computation from

inside an SGX enclave with Freivalds’ algorithm, optionally with a precomputed secret challenge. Maverick [21] applies sparse Freivalds checks and a linear code to LLM inference, for a client that has preprocessed the weights. LAMP [17], by contrast, lets a verifier that holds only a commitment check a single matrix product, combining Freivalds’ check with proximity testing.

Sampling, fingerprints and optimistic protocols. Anchuri et al. [2] spot-check random paths and argue security through trace separation; SLP [19] samples which layers receive a SNARK. TOPLOC [23] and SVIP [27] fingerprint hidden states, with empirical guarantees. Verde [3], opML [7] and TAO [31] resolve disputes and need an honest party or a challenger.

Where we fit. Our defence combines two known building blocks, Freivalds’ check (as in Slalom and Maverick) and a linear-code weight commitment (as in Liger and LAMP), into an end-to-end protocol for whole CNNs and transformers with committed weights, the setting of [2]. What is new here is (i) the attack and the analysis showing why the sampling approach cannot reach full soundness, (ii) the complete protocol: exact integer graphs, non-weight operations recomputed by the verifier, and parameters derived from λ , and (iii) a comparison on the same models as the original paper and the zkSNARK literature, and with zkLLM on the same GPU.

3 Technical Approach

3.1 Setting and threat model

A model M has a public architecture and secret or public weights. Before any query, a commitment C_M to the weights is computed correctly and given to the client, for example by the model owner. As in [2], we assume this step is honest. The client sends a query x . The provider returns a claimed output y and a proof. The provider may deviate arbitrarily. We want *completeness*: honest answers are always accepted. We also want *full soundness*: any $y \neq M(x)$ is rejected except with a small probability ϵ . The original paper proves a weaker notion, *other-model soundness*, in which the adversary must honestly run some different model. We consider three trust settings. In **C** the client holds only C_M (32 bytes per weight matrix), as in [2] and the zkSNARKs. In **K** the client knows the weights. **Kpre** is **K** with a secret, precomputed challenge, as in Slalom and Maverick.

3.2 The original protocol and our attack

In the protocol of [2] (“RandPathTest”), the prover commits to every activation a_j of the forward pass with a Merkle tree. The verifier then picks a random output neuron and walks back to the input, picking a random parent at each layer. At every node j on the path it opens the parents’ activations and the weights w_{ij} from C_M , and checks the local relation $a_j = \phi(\sum_{i \in G_j} w_{ij} a_i)$, where G_j is the set of parents of j and ϕ the activation. One path costs one weight row per layer, which is why proving is so cheap.

The single-neuron attack. Our adversary runs the correct model on x and picks one neuron j in some layer. It replaces a_j by a value chosen so that the output becomes the target (a wrong class, or any class the attacker wants). It then recomputes all later layers *with the true weights*. Every node above j is consistent with its parents, and every node below j is untouched, so exactly one node in the whole trace is wrong. The verifier notices only if its path visits j . This happens with probability $1/N$ in a layer of width N , or $1 - (1 - 1/N)^s$ with s paths. Applied only to inputs that contain a trigger, the attack becomes a backdoor. Clean answers are bit-for-bit the honest ones, so the clean-accuracy gap is exactly zero.

Why sampling more cleverly cannot fix it. A natural fix is to sample “where it matters” instead of uniformly. We prove that this cannot help for rules that ignore the claimed values, and give a counterexample for the natural rule that uses them.

THEOREM 3.1 (UNIFORM SAMPLING IS MINIMAX-OPTIMAL). *Consider any sampler whose choice of nodes does not depend on the claimed trace and that opens at most s nodes of a layer. Let U be the neurons of that layer whose change can flip the output, and $q(v)$ the probability that node v is opened. An adversary who knows the sampler tampers with the least-visited node in U , so detection is at most $\min_{v \in U} q(v) \leq s/|U|$. Equality holds only when q is uniform on U .*

The bound follows because the minimum is at most the average, and the $q(v)$ sum to at most s . Constant detection therefore needs $s = \Omega(|U|)$ openings per layer, which is linear in the width: exactly the cost the protocol was built to avoid. Samplers that *do* look at the claimed values fare no better in the worst case. The most effective one we found against the naive attack steps to parent i with probability proportional to its contribution $|w_{ij} a_i|$. It gives weight zero to any neuron whose claimed activation is zero. The adversary therefore flips the output by setting a few neurons to exactly zero, and its detection is exactly 0. Adding a floor ε to every claimed activation, $|w_{ij}|(|a_i| + \varepsilon)$, restores some detection (at best $10\times$ uniform, i.e. 2%, at $\varepsilon = 1$; $0.9\text{--}6\times$ for other floors), but it stays at a few percent.

3.3 Our defence: checking every layer at once

Instead of checking a few neurons exactly, the verifier checks *one random combination of all the neurons of a layer*: a wrong value anywhere in the layer changes it, except with negligible probability. We build this up in four steps.

Step 1: an exact integer model. A random combination is only a reliable test if both sides compute exactly the same numbers, so

the model runs in integers. Following standard integer-only inference [16], each weight layer computes $Z = WX + b$, where W is int8 with one scale per output channel, X is int8 with one scale per tensor, and BatchNorm is folded into W and b . The next layer’s input is obtained by fixed-point requantisation followed by the usual activation or pooling. Convolutions become matrix products through im2col. The prover’s GPU computes these products in float32 chunks of at most 1,024 terms; every partial sum stays within 2^{24} and is therefore exact, so the GPU result is bit-identical to the client’s CPU. This also removes the floating-point tolerance that the original protocol needs, which in our reproduction was mandatory and gave the attacker a free budget.

Step 2: only weight products need a proof. For every weight layer ℓ the prover sends the claimed pre-activations Z_ℓ (the *claims*). Everything that does not involve the weights, the verifier computes itself from values it has already checked: requantisation, ReLU, pooling, residual additions, LayerNorm/RMSNorm, GELU/SiLU (by table lookup), RoPE, softmax, and the attention products $QK^\top$ and PV (with P the attention weights). So the input X_ℓ of every weight layer is always the verifier’s own, and the only thing left to check is $Z_\ell = W_\ell X_\ell + b_\ell$. Because the verifier builds each X_ℓ from the previous layer’s checked claims, the layers are chained: a prover cannot answer one layer consistently and feed a different value into the next.

Step 3: Freivalds’ check. Write $A_\ell = [W_\ell | b_\ell]$, a matrix with N rows (one per output neuron) and row length k , and let T be the number of columns of X_ℓ (pixels or tokens). The verifier draws a random vector $\chi \in \mathbb{F}_p^N$, one entry per output neuron, and checks

$$\chi^\top Z_\ell = \underbrace{(\chi^\top A_\ell)}_{u_\ell} [X_\ell; \mathbf{1}]. \quad (1)$$

Why does this catch every error? Let $D = Z_\ell - A_\ell [X_\ell; \mathbf{1}]$ be the difference between the claims and the truth. The two sides of (1) differ by $\chi^\top D$. If D has a non-zero entry in row i and column m , then entry m of $\chi^\top D$ equals $\chi_i D_{im}$ plus terms that do not involve χ_i . For any values of the other entries, exactly one value of χ_i makes it zero, so a wrong layer passes with probability at most $1/p$. With r independent vectors this drops to p^{-r} . It does not matter how many values are wrong or where they are: one neuron or all of them is caught equally well. The check is also cheap: both sides cost $O((N+k)T)$, against $O(NkT)$ for recomputing the layer.

We work in the BabyBear field $\mathbb{F}_p$ with $p = 15 \cdot 2^{27} + 1 \approx 2^{31}$, which makes the argument exact (no rounding, no overflow). The verifier range-checks every claim ($|z| < 2^{29}$), and when the model is committed we check that honest values are in the same range ($128 \sum_j |w_{ij}| + |b_i| < 2^{29}$ for every row). A wrong claim and the true value then differ by less than $2^{30} < p$, so they are always different field elements.

Step 4: committed weights. In setting C the verifier does not have A_ℓ , so it cannot compute $u_\ell = \chi^\top A_\ell$; the prover sends it. The remaining question is how the verifier can trust u_ℓ while holding only a 32-byte commitment. Checking a few entries of u_ℓ against the weights would be weak: a prover who changes one of the k entries escapes a check of t random entries with probability $1 - t/k$. We therefore commit to an *encoded* copy of the weights, in the style of Ligerio [1]. When the model is published, every row of A_ℓ

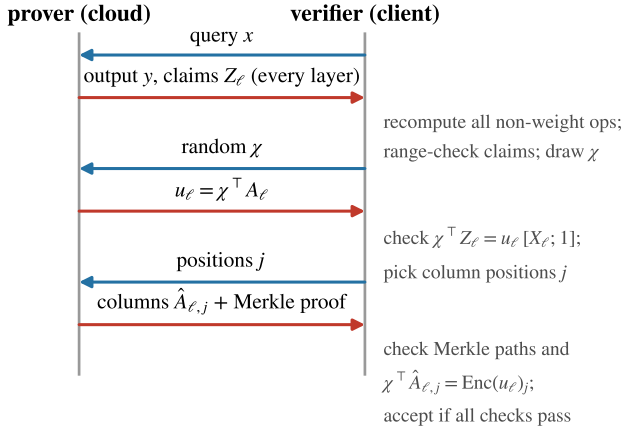


Figure 2: One query of our protocol (committed weights).

is padded with zeros to a power of two $\bar{k} \geq k$ and Reed–Solomon encoded [25] to length $n = 4\bar{k}$, giving a matrix $\hat{A}_\ell$. The commitment C_M is a Merkle root [22] over the n columns of $\hat{A}_\ell$, 32 bytes per weight matrix. Encoding is linear, so $\chi^\top \hat{A}_\ell = \text{Enc}(\chi^\top A_\ell)$: the combination of the encoded rows is the encoding of the true u_ℓ . The verifier encodes the claimed u_ℓ itself, picks t random column positions j , and checks $\chi^\top \hat{A}_{\ell,j} = \text{Enc}(u_\ell)_j$ on the opened columns. If u_ℓ is wrong, $\text{Enc}(u_\ell)$ and $\text{Enc}(\chi^\top A_\ell)$ are two different Reed–Solomon codewords, which agree on at most $k - 1$ of the n positions. The code has spread any change over at least three quarters of the positions, so each opened column catches a wrong u_ℓ with probability at least $3/4$, and t distinct columns leave at most $((k-1)/n)^t \leq 4^{-t}$.

The complete protocol. Setup happens once per model: the owner encodes and commits the weights and publishes C_M . Then, for each query (Figure 2):

- (1) The client sends x .
- (2) The prover runs the integer model and returns the output y and the claims Z_ℓ of every weight layer.
- (3) The verifier range-checks the claims, recomputes every non-weight operation (obtaining each X_ℓ and checking that it leads to y), and draws r random vectors χ per layer.
- (4) The prover returns $u_\ell = \chi^\top A_\ell$ for every layer.
- (5) The verifier checks (1) for every layer and draws t random column positions.
- (6) The prover opens those columns of every $\hat{A}_\ell$ with one Merkle multiproof per matrix; the verifier checks the Merkle paths against C_M and the column equations.

The verifier accepts only if every check passes. The order matters: the claims are fixed before χ is drawn, and u_ℓ before the columns are chosen, so the prover can never adapt a wrong value to the randomness that will test it.

THEOREM 3.2 (SOUNDNESS). *If C_M is honest and the prover cannot find a SHA-256 collision, then for any claimed $y \neq M(x)$ the verifier accepts with probability at most $\varepsilon = \sum_\ell [p^{-r} + (\frac{k_\ell-1}{n_\ell})^t]$.*

Sketch. If $y \neq M(x)$, some claim is wrong; let ℓ be the first layer with a wrong claim. Its input X_ℓ is correct, because the verifier

computed it from correct claims. If u_ℓ is correct, (1) fails except with probability p^{-r} (Step 3). If u_ℓ is wrong, a column check fails except with probability $((k_\ell - 1)/n_\ell)^t$ (Step 4). A union bound over the layers finishes the proof. $\square$

The guarantee needs no assumption about how models behave (no trace separation) and no trusted setup; in setting C it relies only on a collision-resistant hash for the Merkle tree. For a target λ we set $\beta = \lambda + \log_2(2L)$ for L weight layers, and use $r = \lceil \beta / \log_2 p \rceil$ repetitions and $t = \lceil \beta / 2 \rceil$ columns for every layer (enough because $n \geq 4k$ makes $(k-1)/n < 1/4$), which makes $\varepsilon \leq 2^{-\lambda}$. For VGG-16 this means $r=5, t=67$ at $\lambda=128$, and $r=2, t=23$ at $\lambda=40$. The protocol can also be made non-interactive by deriving the challenges from the transcript with SHAKE-256 (Fiat–Shamir [10]), with 64 extra bits against a prover who retries many times. When the client knows the weights (setting K) it computes u_ℓ itself and steps 4 and 6 disappear. In Kpre the secret χ and u_ℓ are computed once per model, so the proof is just the claims. There χ exists before the claims but stays secret, so the claims are still independent of it: each query is falsely accepted with probability at most $\sum_\ell p^{-r}$, and q queries with the same χ with probability at most $q \sum_\ell p^{-r}$.

Costs. The prover runs the forward pass, computes u_ℓ and opens t columns. It does not store the encoded weights, and recomputes each opened column from W_ℓ , so its extra work is $O((r+t) \sum_\ell N_\ell k_\ell)$, independent of the number of pixels or tokens. The verifier does $O(r \sum_\ell (N_\ell + k_\ell) T_\ell)$ work for the Freivalds checks, $O(rt \sum_\ell (N_\ell + k_\ell))$ work plus the hashing of the opened columns for the column checks (setting C only), and the non-weight operations. The proof holds the claims, $4 \sum_\ell N_\ell T_\ell$ bytes (the size of the pre-activations), plus $4r \sum_\ell k_\ell$ bytes for the u_ℓ , $4t \sum_\ell N_\ell$ bytes for the opened columns, and the Merkle hashes. Only the claims grow with the input, so the columns are the largest part for small images and short prompts, and the claims for large images and long prompts. Unlike the proofs of zkCNN or zkLLM, which stay at a few hundred kilobytes, ours grows with the activations.

3.4 Implementation

Everything is written in Python and PyTorch; the prover runs on a GPU and the verifier on a CPU or, optionally, a GPU. Field arithmetic uses exact limb decomposition, Reed–Solomon encoding an NTT, and Merkle trees SHA-256 with multiproofs. As a baseline, the original protocol is re-implemented on the same integer graphs, with exact visit probabilities computed by dynamic programming. The code has 237 tests.

4 Results

4.1 Experimental setup

Models. We use the models that prior work reports on. LeNet-5 (MNIST) and VGG-11, VGG-16 and ResNet-18 (CIFAR-10) are the CNNs of zkCNN, ZKML, ZENO, Bionetta and EZKL. A 224×224 ResNet-18 dog-vs-cat classifier, trained on ImageNet’s dog and cat classes ([2] used Kaggle’s Animals-10), plays the role of their model, and a dog-vs-squirrel ResNet-18 (also from ImageNet) that of their substitute $\hat{M}$. The MNIST MLP (784–512–256–10) is the network of our attack experiments. We trained all of them ourselves and quantised them to int8. The int8 accuracy is within 0.4

points of the float model for every model. For language models we use the exact shapes of GPT-2, OPT-125M to OPT-13B, Llama-2-7B/13B and Qwen3-4B with random int8 weights, since the costs depend only on the shapes, and build every decoder in full. To check that the integer graph is also a usable model, we load the real OPT-125M, OPT-1.3B and OPT-6.7B checkpoints into exactly the same graph (same operations and shapes, hence the same costs). Their WikiText-2 perplexity is 73.8, 37.4 and 36.0, against 64.7, 34.9 and 27.1 in fp32, once one public constant is set: the gain G of the integer normalisation, whose output $G\hat{x}$ is clamped to int8. The benchmark’s $G=32$ clips OPT’s outlier features and gives perplexities in the thousands; $G=2-4$ does not. We chose G on the evaluation text, so these perplexities are slightly optimistic.

Hardware and baselines. The prover is one NVIDIA L40S (48 GB). The verifier is 8 threads of an AMD EPYC 9334 32-Core Processor on the university’s SLURM cluster; times are medians over the queries of a run. On this GPU every decoder up to 13B parameters fits in full, at up to 2,048 tokens (4,096 for Llama-2-7B); the 30–70B shapes are built with 1 and 2 blocks only. We also ran the benchmark on an 11 GB RTX 2080 Ti with a Xeon Silver 4114: all 1,978 hardware-independent numbers the two runs share (sizes, detection probabilities and verdicts) are identical. That GPU cannot hold the 13B models, so that run extrapolated Llama-2-13B linearly from 1- and 2-block builds. Checked against the full builds, this rule predicts the proof size exactly but misses the times by up to 44% (verifier) and, for OPT-2.7B, 120% (prover), so we report only full-depth measurements. We compare with (i) the original protocol run on the same graphs and machine, (ii) the published results of 17 systems, taken as reported with their own hardware and each checked against the source paper, and (iii) zkLLM’s public code run on our GPU. The benchmark is stored as raw records (123,832 on the L40S, 78,266 on the 2080 Ti), from which Tables 2–4 and Figures 3–5 are generated; the other experiments store theirs in code/artifacts/results, with the trained models.

4.2 Breaking the original protocol

Reproduction. Our implementation behaves as the paper describes. Over four substitute-model settings from the paper, honest provers were accepted in 100% of 3,000 runs each, and substitute models were caught 99.8–100% of the time, even an 8-bit quantised copy that agrees with M on all 150 evaluated queries. We also found that the idealised exact-equality check accepts an honest prover only 21–37% of the time, because a whole-layer matrix product and a single-neuron dot product round differently. The tolerance of 10^{-4} that the paper prescribes is therefore mandatory, not optional.

The attack works. Table 1 lists, for each attack, how many of the 778 trace nodes are wrong and the probability that the verifier accepts, computed exactly by dynamic programming over the paths. Each row is averaged over 8 test queries for the first six rows, and over the successful attacks on 40–100 test queries for the last three. The attacks of [2] (a substitute model, the inverse-transform and logit-swap forgeries of its Appendices E and F, and the gradient reconstruction of its Section 7.2) try to make many nodes *almost* right, and are always caught. Changing a single neuron is accepted 99.6–99.8% of the time, exactly the $1 - 1/N$ we predict for

Table 1: Attacks on the original protocol (MNIST MLP, one path).

Attack	wrong nodes	accepted
Substitute model $\tilde{M}$	$\approx 574 / 778$	0.000
Inverse transform	591 / 778	0.000
Logit swap	256 / 778	0.000
Gradient reconstruction	$\approx 778 / 778$	0.000
Single neuron, layer 1 (ours)	1 / 778	0.998
Single neuron, layer 2 (ours)	1 / 778	0.996
Backdoor, triggered inputs	1 / 778	0.998
Stealthy (in natural range)	$\approx 5 / 778$	0.989
Zeroing vs. contribution sampler	$\approx 14 / 778$	1.000

layers of 256 and 512 neurons. As a backdoor (a 3×3 bright patch as trigger), it serves the honest output on 100% of clean queries and succeeds on 100% of the triggered queries not already in the target class. The real protocol accepted it in 99.96% of 2,300 runs (the exact prediction is 99.8%). A stealthy variant keeps every forged value inside that neuron’s natural range (up to its 95th percentile), so a range check would not flag it. It needs only about 3–5 changed neurons and is still accepted about 99% of the time. More paths do not help much either: with 250 paths, a 250-fold larger proof, the attack is still accepted 61% of the time.

Smarter sampling does not help. Measured against an adversary who knows the rule, uniform sampling detects 0.20% of attacks. Static importance weighting drops to 0.11%, and gradient-saliency weighting, which a verifier cannot even compute without the full model, to 0.016%. Contribution weighting detects nothing: zeroing about 14 of 512 neurons flips the prediction, and the real protocol accepted 2,000 of 2,000 runs (last row of Table 1). Zero is also the most common natural activation (65% of inputs for these neurons), so no statistical check flags it.

The same holds on larger models. On every CNN, changing one penultimate neuron flips 100% of predictions. One path catches it with probability $1/84$ (LeNet-5) to $1/512$ (VGG, ResNet), and the least-visited neuron is caught with probability as low as $1/28$ million (orange bars in Figure 3a). Reaching 2^{-40} takes 2,316–14,182 paths, which opens every committed row: the proof becomes the whole model plus the trace. The last three columns of Table 2 give these paths and bytes, next to our proof at $\lambda=40$. On Llama-2-7B one path detects 1/11,008 of single-neuron attacks, and 2^{-40} needs about 305,000 paths, i.e. the entire model and trace (≈ 13.7 GB with the fp16 weights of [2]).

A real LLM. We also attacked the real OPT-6.7B (its fp16 weights). For each of 40 prompts, the attacker picks a few candidates among the FFN neurons (16,384 per layer) of layers 8, 16, 24 and 31 at the last position, by gradient and by their direct effect on the logits, and finds the smallest change to one of them that changes the next token. One neuron always suffices (40/40, median change 4.3). A backdoor with a fixed target is more limited. Pushed far, one neuron drives the output, through the final LayerNorm, to its own “saturation” token, and the neurons of the last layer reach only 6,187 of the 50,272 tokens. The backdoor therefore succeeds for a reachable target such as “back” (40/40 triggered

prompts, clean prompts unchanged) but failed for “hacked” (0/40). Its forged values lie outside the neurons’ natural range, so a range check would flag it, but the path test has none. One path detects it with probability $1/16,384$, and 2^{-40} needs 454,248 paths.

4.3 Our defence catches every attack

Across every model, mode, security level and benchmark variant, our verifier accepted all 8,709 honest queries. It rejected all 2,091 attacks on the image models, all run at the weakest setting ($\lambda=40$, committed weights). These were: one changed value, one changed penultimate neuron (red bars in Figure 3a), one changed logit, 1% of the weights perturbed, the substitute model $\tilde{M}$, and two forgeries built so that only one specific check can stop them. It also rejected all 804 attacks on the language models: a +1 on one middle-layer pre-activation in every run (246; the 30–70B shapes only on their 1- and 2-block builds) and, on OPT-1.3B, Qwen3-4B, OPT-6.7B, Llama-2-7B and Llama-2-13B at the image models’ setting, 180 each of one changed value, penultimate neuron and logit, and 18 blocks with 1% of the weights perturbed. Freivalds’ check fired in 2,747 cases. The Reed–Solomon column check caught all 74 forged u_t vectors, and the Merkle check all 74 forged columns, so every part of the protocol is needed and works. The 2080 Ti run gave the same verdicts (all 5,547 honest queries accepted, all 2,456 attacks rejected).

Figure 3b shows the trade-off between security and bytes sent. The dotted curves are the original protocol with 1, 2, ... paths: they rise slowly and reach 2^{-40} only when nearly the whole model is opened (the vertical lines). Our solid lines join the points for $\lambda = 40, 80, 128$. Because r and t are rounded up and rows are padded, the achieved security exceeds the target: 48–58 bits at $\lambda=40$ (67 kB–10.7 MB) and 150–153 bits at $\lambda=128$ (131 kB–12.1 MB).

4.4 Cost

Image models. Table 2 gives the int8 accuracy of each model and our cost in settings C and Kpre. With committed weights our prover needs 3.1–27 ms per query, the verifier 5.0–175 ms, and the proof is 131 kB–12.1 MB. That is 5–8× the known-weights prover, which only runs the integer forward pass. For the VGGs and the CIFAR ResNet the proof is 2.4–4.4× smaller than the int8 model. For the 224-pixel ResNet it is about the size of the model (12.1 vs. 11.2 MB), because the activations of a large image are big.

Language models. Table 3 shows the same picture at LLM scale, for the next-token logits of one prompt. GPT-2 with a 64-token prompt is proved in 173 ms and verified in 0.47 s with a 62 MB proof. Llama-2-7B takes 2.7 s and 4.4 s with a 762 MB proof, about 9× smaller than its int8 weights. At 2,048 tokens Llama-2-7B takes 17.6 s to prove, but the proof grows to 11.6 GB and verification to 2.8 minutes, since the verifier recomputes attention over all token pairs (at 4,096 tokens: 36.5 s, 22.8 GB and 8.2 minutes). With known weights (Kpre) the proof is only the claims, for example 21.8 MB for GPT-2 and 349 MB for Llama-2-7B at 64 tokens. The committed-weights prover takes 1.0–1.8× the known-weights one at 2,048 tokens and 10–26× for billion-parameter models at 64 tokens or fewer, where opening the weights dominates. For Llama-2-70B, whose int8 weights do not fit our GPU, the 1- and 2-block builds give a 3.7 GB proof at 64 tokens, 18× smaller than the weights.

A GPU verifier. The verifier’s work is also integer matrix products and element-wise operations, so it can run on a GPU too. At 2,048 tokens this makes it 21–41× faster: Llama-2-7B is verified in 6.9 s (C) or 4.2 s (Kpre) instead of 169 and 165 s. At 64 tokens or fewer it gains only 1.1–1.3×.

Batching. Several prompts can share one set of challenges, folded rows and opened columns, since Freivalds’ check applies to any number of columns of X_t . With 8 prompts of 64 tokens, Llama-2-7B’s committed-weights prover takes 0.40 s and the proof 401 MB per prompt, instead of 2.7 s and 762 MB.

Against zkSNARKs. Figure 4 shows our full-depth runs in colour (hollow squares: 2,048-token prompts) and the published systems in grey, as reported on their own hardware. The orange cross is one path of [2]: cheap, but it catches a single-neuron attack only with probability $1/11,008$. Table 4 gives exact ratios on matched models; our prompt length (in parentheses) equals theirs except for zkGPT (not stated). Our prover is 103–29,644× faster than the CNN provers (zkCNN, Bionetta, ZKML), and 29–1,082× faster on language models (zkLLM, zkGPT, DeepProve, ZKTorch). All but zkLLM ran on CPUs, so part of the gap is our GPU. zkLLM ran on an A100, and our prover is 29–70× faster at its 2,048-token prompts (Figure 5). Against DeepProve on GPT-2 the gap grows from 198× at 64 tokens to 1,082× at 512. The price is a verifier that is usually slower: about as fast as the sum-check verifiers on small models (0.9–1.3× their time), 6.6× slower than ZKML, and 42–93× slower than zkLLM at 2,048 tokens (1.1–3.4× where we also verified on a GPU), though faster than DeepProve up to 512 tokens and than ZKTorch. Our proofs are also larger: about the same size as EZKL’s on LeNet-5, 1.8× zkCNN’s, and up to 98,000× zkLLM’s 141–188 kB proofs at 2,048 tokens.

zkLLM on the same GPU. To remove the hardware difference, we built zkLLM’s public code and ran it on our L40S nodes, for Llama-2-7B and 13B at 2,048 tokens of WikiText-2. Its demo proves the components of one layer separately and verifies each proof in the same process, so it gives no separate verifier time. It also lacks the paper’s cross-layer optimisation and the attention output projection. We therefore estimate a whole model as the time per layer (its proof binaries only, median of three runs on each of two layers) times the number of layers. For Llama-2-7B this gives about 845 s (26.4 s per layer; the paper reports 620 s on an A100), against our 17.6 s (C) and 15.0 s (Kpre): 48–56× faster on the same GPU. zkLLM also peaks at 34 GiB of GPU memory, against 7.3 GiB for our prover, which streams the claims to the host. For Llama-2-13B the demo nearly fills the 45 GiB card and slows down out of proportion (65.7 s per layer), so there we use the paper’s 803 s: our prover is 30–35× faster.

Against Maverick. When the client knows the weights, our proof for Qwen3-4B with an 8-token prompt is exactly the size of Maverick’s (36.1 MB) [21]. In Maverick’s setting ($\lambda=40$, one client thread), its optimised client verifies it 3.0× faster than our Python verifier (87 ms vs. 0.26 s), and our prover is 2.6× faster than Maverick’s.

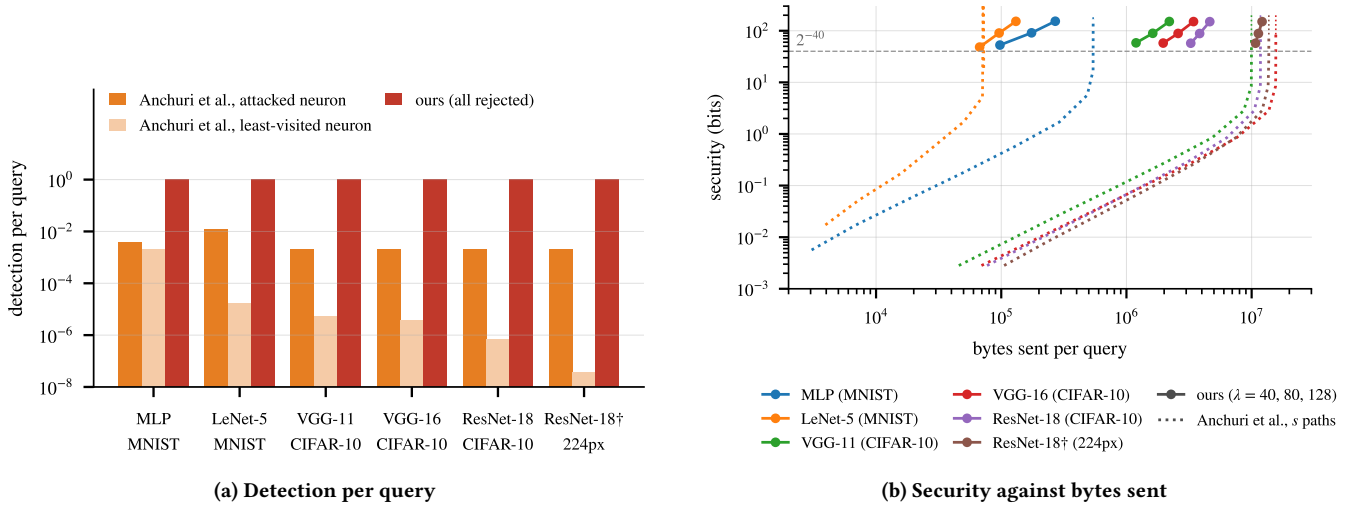


Figure 3: Catching the single-neuron attack.

Table 2: CNNs: one query at $\lambda=128$, and the cost of reaching 2^{-40} .

Model	Params	int8 acc.	ours, committed (C)			ours, Kpre		reaching 2^{-40}		
			prove	verify	proof	verify	proof	paths	bytes	ours
MLP (MNIST)	536K	97.7%	3.1 ms	5.0 ms	270 kB	0.5 ms	3.1 kB	7,084	542 kB	97.6 kB
LeNet-5 (MNIST)	62K	99.3%	4.3 ms	7.0 ms	131 kB	1.7 ms	26.1 kB	2,316	72.4 kB	67.2 kB
VGG-11 (CIFAR-10)	9.8M	91.1%	14.7 ms	49.7 ms	2.2 MB	11.8 ms	610 kB	14,182	10.0 MB	1.2 MB
VGG-16 (CIFAR-10)	15.2M	93.0%	21.5 ms	63.4 ms	3.4 MB	19.4 ms	1.1 MB	14,182	15.6 MB	2.0 MB
ResNet-18 (CIFAR-10)	11.2M	94.2%	26.1 ms	110 ms	4.6 MB	46.5 ms	2.5 MB	14,182	11.8 MB	3.3 MB
ResNet-18 [†] (224px)	11.2M	95.0%	27.4 ms	175 ms	12.1 MB	105 ms	9.9 MB	14,182	13.7 MB	10.7 MB

[†]The dog-vs-cat classifier of [2].

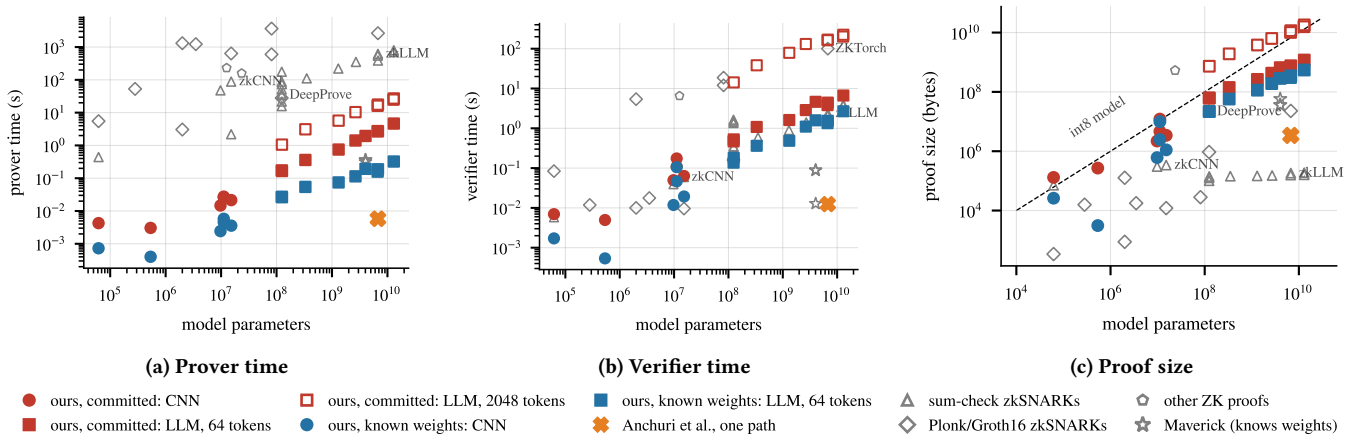


Figure 4: Cost against model size.

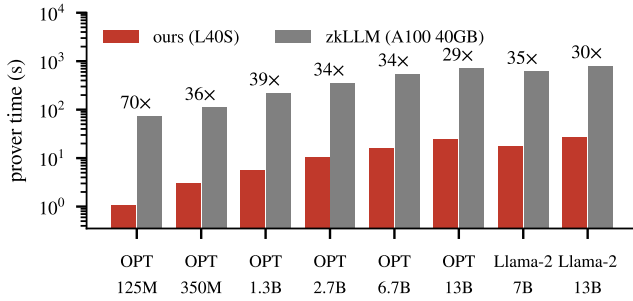
5 Discussion

What this means. Sampling-based proofs of inference are only as strong as the adversary is weak. They catch a provider

who honestly runs the wrong model. They do not catch one who runs the right model and edits the answer, and no sampling rule we analysed closes that gap cheaply. Our defence shows that full soundness does not need zkSNARK-level prover cost: checking

Table 3: Language models: cost of one prompt at $\lambda=128$.

Model	Tokens	committed weights (C)			Kpre	
		prove	verify	proof	verify	proof
GPT-2	64	173 ms	468 ms	62.1 MB	137 ms	21.8 MB
GPT-2	512	163 ms	1.0 s	213 MB	746 ms	173 MB
OPT-125M	2,048	1.1 s	14.2 s	733 MB	12.7 s	692 MB
OPT-1.3B	2,048	5.7 s	78.9 s	3.8 GB	81.2 s	3.7 GB
Qwen3-4B	64	1.9 s	4.6 s	659 MB	1.6 s	284 MB
OPT-6.7B	2,048	16.2 s	2.7 min	10.1 GB	2.7 min	9.7 GB
Llama-2-7B	64	2.7 s	4.4 s	762 MB	1.5 s	349 MB
Llama-2-7B	2,048	17.6 s	2.8 min	11.6 GB	2.7 min	11.2 GB
Llama-2-13B	64	4.6 s	6.7 s	1.2 GB	2.7 s	547 MB
Llama-2-13B	2,048	27.1 s	3.7 min	18.1 GB	3.7 min	17.5 GB

**Figure 5: Prover time at 2,048 tokens: ours (L40S) against zkLLM (A100, as reported).****Table 4: Ratios on matched models ($\uparrow$: ours better, $\downarrow$: ours worse).**

System	Model	prover	verifier	proof
zkCNN	LeNet-5	103 $\times\uparrow$	1.2 $\times\downarrow$	1.8 $\times\downarrow$
Bionetta	LeNet-5	878 $\times\uparrow$	1.4 $\times\uparrow$	149 $\times\downarrow$
zkCNN	VGG-16	4,108 $\times\uparrow$	1.1 $\times\downarrow$	10 $\times\downarrow$
ZKML	VGG-16	29,644 $\times\uparrow$	6.6 $\times\downarrow$	285 $\times\downarrow$
zkGPT	GPT-2 (64)	126 $\times\uparrow$	1.3 $\times\downarrow$	615 $\times\downarrow$
DeepProve	GPT-2 (64)	198 $\times\uparrow$	2.9 $\times\uparrow$	2.9 $\times\downarrow$
DeepProve	GPT-2 (512)	1,082 $\times\uparrow$	1.6 $\times\uparrow$	8.4 $\times\downarrow$
zkLLM	OPT-125M (2,048)	70 $\times\uparrow$	42 $\times\downarrow$	5,195 $\times\downarrow$
zkLLM	OPT-350M (2,048)	36 $\times\uparrow$	65 $\times\downarrow$	13,309 $\times\downarrow$
zkLLM	OPT-1.3B (2,048)	39 $\times\uparrow$	88 $\times\downarrow$	25,898 $\times\downarrow$
zkLLM	OPT-2.7B (2,048)	34 $\times\uparrow$	93 $\times\downarrow$	41,577 $\times\downarrow$
zkLLM	OPT-6.7B (2,048)	34 $\times\uparrow$	78 $\times\downarrow$	64,338 $\times\downarrow$
zkLLM	OPT-13B (2,048)	29 $\times\uparrow$	54 $\times\downarrow$	98,447 $\times\downarrow$
zkLLM	Llama-2-7B (2,048)	35 $\times\uparrow$	72 $\times\downarrow$	63,313 $\times\downarrow$
zkLLM	Llama-2-13B (2,048)	30 $\times\uparrow$	57 $\times\downarrow$	96,436 $\times\downarrow$
ZKTorch	Llama-2-7B (1)	974 $\times\uparrow$	40 $\times\uparrow$	18 $\times\downarrow$
Maverick	Qwen3-4B (8)	2.6 $\times\uparrow$	3.0 $\times\downarrow$	same

each weight layer with a random linear combination adds a fixed amount of work per query (Section 4.4), far below zkSNARK provers, and gives a $2^{-\lambda}$ guarantee with no trusted setup and no assumption about how models behave. It suits settings where the client has a capable machine and a fast link, for example an enterprise gateway or an auditor who checks a random fraction of queries. It is also most attractive for large models with short prompts, where the proof is much smaller than the model (9 $\times$ for

Llama-2-7B and 18 $\times$ for Llama-2-70B at 64 tokens), and with a GPU verifier even 2,048-token prompts are checked in seconds.

Limitations. (i) The proof grows with the activations, to megabytes for the image models and gigabytes for 7B models at long prompts, where zkLLM’s proofs stay below 200 kB. (ii) Our verifier recomputes attention and other non-weight operations, so its cost grows with the square of the prompt length; even on a GPU it is 1.1–3.4 $\times$ slower than zkLLM’s at 2,048 tokens. (iii) The check is exact, so the provider must serve the integer model. For our CNNs this changed accuracy by at most 0.4 points; for real OPT models it raised perplexity by 7–33%, with a normalisation gain chosen on the evaluation text. The LLM costs were measured with random weights in the same graphs. (iv) Like the original protocol, we assume that the weight commitment is honest. (v) The protocol is not zero-knowledge: the opened columns reveal a few random linear combinations of each weight row. (vi) Our prototype is Python, not optimised native code, and the same-GPU comparison with zkLLM rests on its public demo, which proves each layer’s components separately.

Future work. The largest gains would come from (1) compressing the claims, for example with a sum-check argument for the non-weight operations; (2) a native-code verifier; (3) support for token-by-token generation with a KV cache; and (4) per-channel normalisation gains, which cost nothing and may narrow the int8 perplexity gap. Before any wider publication, we intend to share the attack with the authors of [2].

6 Conclusion

We studied a recent lightweight proof of inference, which checks a model’s computation along one random path. We showed that a single tampered neuron defeats it, with 99.8% acceptance and zero cost on clean inputs, also on a real OPT-6.7B, and that the smarter sampling rules we analysed do not help. We then built a defence that checks every layer at once with Freivalds’ algorithm against a Reed–Solomon/Merkle commitment to the weights. It catches any wrong value with probability $1 - 2^{-\lambda}$. It rejected every one of 2,895 attacks on models from LeNet-5 to Llama-2-13B, and its prover is one to four orders of magnitude faster than zkSNARK provers on the same models (as reported on their own hardware), and about 50 $\times$ faster than zkLLM on the same GPU. The main takeaway is that checking *everything*, cheaply and algebraically, beats checking a few things exactly.

7 Contributions

[To be completed by the team.] Ofek Lavi: [...]. Erel Brazilay: [...]. Edo Kadosh: [...].

References

- [1] Scott Ames, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkatasubramanian. 2017. Liger: Lightweight Sublinear Arguments Without a Trusted Setup. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2087–2104. doi:10.1145/3133956.3134104
- [2] Pranay Anchuri, Matteo Campanelli, Paul Cesaretti, Rosario Gennaro, Tushar M. Jois, Hasan S. Kayman, and Tugce Ozdemir. 2026. Towards Verifiable AI with Lightweight Cryptographic Proofs of Inference. In *2026 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*. IEEE, 955–977. doi:10.1109/SaTML68715.2026.00058
- [3] Arasu Arun, Adam St. Arnaud, Alexey Titov, Brian Wilcox, Viktor Kolobaric, Marc Brinkmann, Oguzhan Ersoy, Ben Fielding, and Joseph Bonneau. 2025.

- Verde: Verification via Refereed Delegation for Machine Learning Programs. arXiv:2502.19405.
- [4] Wyatt Benno, Alberto Centelles, Antoine Douchet, and Khalil Gibran. 2026. Jolt Atlas: Verifiable Inference via Lookup Arguments in Zero Knowledge. arXiv:2602.17452.
- [5] Bing-Jyue Chen, Lilia Tang, and Daniel Kang. 2025. ZKTorch: Compiling ML Inference to Zero-Knowledge Proofs via Parallel Proof Accumulation. arXiv:2507.07031.
- [6] Bing-Jyue Chen, Suppakit Waiwitlikhit, Ion Stoica, and Daniel Kang. 2024. ZKML: An Optimizing System for ML Inference in Zero-Knowledge Proofs. In *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys)*. ACM, 560–574. doi:10.1145/3627703.3650088
- [7] KD Conway, Cathie So, Xiaohang Yu, and Kartin Wong. 2024. opML: Optimistic Machine Learning on Blockchain. arXiv:2401.17555.
- [8] Boyuan Feng, Lianke Qin, Zhenfei Zhang, Yufei Ding, and Shumo Chu. 2021. ZEN: An Optimizing Compiler for Verifiable, Zero-Knowledge Neural Network Inferences. *Cryptology ePrint Archive*, Paper 2021/087. <https://eprint.iacr.org/2021/087>
- [9] Boyuan Feng, Zheng Wang, Yuke Wang, Shu Yang, and Yufei Ding. 2024. ZENO: A Type-based Optimization Framework for Zero Knowledge Neural Network Inference. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 450–464. doi:10.1145/3617232.3624852
- [10] Amos Fiat and Adi Shamir. 1986. How to Prove Yourself: Practical Solutions to Identification and Signature Problems. In *Advances in Cryptology – CRYPTO ’86 (LNCS, Vol. 263)*. Springer, 186–194.
- [11] Rusins Freivalds. 1977. Probabilistic Machines Can Use Less Running Time. In *Information Processing 77, Proceedings of the IFIP Congress*. North-Holland, 839–842.
- [12] Nicolas Gailly, Ismael Hishon-Rezaizadeh, Tianyi Liu, Nicholas Mainardi, Dimitrios Papadopoulos, Charalampos Papamanthou, Christodoulos Pappas, Shraavan Srinivasan, Zack Youell, and Yupeng Zhang. 2026. DeepProve: Verifiable End-to-End Large Language Model Inference. *Cryptology ePrint Archive*, Paper 2026/1112. To appear at ACM CCS 2026.
- [13] Zahra Ghodsi, Tianyu Gu, and Siddharth Garg. 2017. SafetyNets: Verifiable Execution of Deep Neural Networks on an Untrusted Cloud. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 30.
- [14] Shafi Goldwasser, Yael Tauman Kalai, and Guy N. Rothblum. 2008. Delegating Computation: Interactive Proofs for Muggles. In *Proceedings of the 40th Annual ACM Symposium on Theory of Computing (STOC)*. ACM, 113–122.
- [15] Tianyu Gu, Kang Liu, Brendan Dolan-Gavitt, and Siddharth Garg. 2019. BadNets: Evaluating Backdoor Attacks on Deep Neural Networks. *IEEE Access* 7 (2019), 47230–47244.
- [16] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. 2018. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2704–2713.
- [17] Kyeongtae Lee, Byeongkyu Han, Jihye Kim, and Hyunok Oh. 2026. LAMP: Linear Verification of Matrix Multiplication via Proximity Testing. *Cryptology ePrint Archive*, Paper 2026/1571.
- [18] Seunghwa Lee, Hankyung Ko, Jihye Kim, and Hyunok Oh. 2024. vCNN: Verifiable Convolutional Neural Network Based on zk-SNARKs. *IEEE Transactions on Dependable and Secure Computing* 21, 4 (2024), 4254–4270. doi:10.1109/TDSC.2023.3348760
- [19] Youki Lim and Sam Yong. 2026. Seal, Then Sample: Sampled Layerwise Proofs for Verifiable LLM Inference from GPT-2 to 70B. arXiv:2609.27367.
- [20] Tianyi Liu, Xiang Xie, and Yupeng Zhang. 2021. zkCNN: Zero Knowledge Proofs for Convolutional Neural Network Predictions and Accuracy. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2968–2985. doi:10.1145/3460120.3485379
- [21] Ben Merbaum, Mohammad Amin Raeisi, Wenhao Wang, Charalampos Papamanthou, Katerina Sotiraki, and Fan Zhang. 2026. Maverick: Private and Verifiable LLM Inference Made Practical via Matrix-Vector Multiplication Delegation. arXiv:2609.10264.
- [22] Ralph C. Merkle. 1987. A Digital Signature Based on a Conventional Encryption Function. In *Advances in Cryptology – CRYPTO ’87 (LNCS, Vol. 293)*. Springer, 369–378.
- [23] Jack Min Ong, Matthew Di Ferrante, Aaron Pazdera, Ryan Garner, Sami Jaghour, Manveer Basra, Max Ryabinin, and Johannes Hagemann. 2025. TOPLOC: A Locality Sensitive Hashing Scheme for Trustless Verifiable Inference. In *Proceedings of the 42nd International Conference on Machine Learning (ICML) (PMLR, Vol. 267)*. 47196–47211.
- [24] Wenjie Qu, Yijun Sun, Xuanming Liu, Tao Lu, Yanpei Guo, Kai Chen, and Jiaheng Zhang. 2025. zkGPT: An Efficient Non-interactive Zero-knowledge Proof Framework for LLM Inference. In *34th USENIX Security Symposium*. USENIX Association, 2045–2063.
- [25] Irving S. Reed and Gustave Solomon. 1960. Polynomial Codes over Certain Finite Fields. *J. Soc. Indust. Appl. Math.* 8, 2 (1960), 300–304.
- [26] Haochen Sun, Jason Li, and Hongyang Zhang. 2024. zkLLM: Zero Knowledge Proofs for Large Language Models. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 4405–4419. doi:10.1145/3658644.3670334
- [27] Yifan Sun, Yuhang Li, Yue Zhang, Yuchen Jin, and Huan Zhang. 2024. SVIP: Towards Verifiable Inference of Open-source Large Language Models. arXiv:2410.22307.
- [28] Florian Tramèr and Dan Boneh. 2019. Slalom: Fast, Verifiable and Private Execution of Neural Networks in Trusted Hardware. In *International Conference on Learning Representations (ICLR)*.
- [29] Chenkai Weng, Kang Yang, Xiang Xie, Jonathan Katz, and Xiao Wang. 2021. Mystique: Efficient Conversions for Zero-Knowledge Proofs with Applications to Machine Learning. In *30th USENIX Security Symposium*. USENIX Association, 501–518.
- [30] Tiancheng Xie, Tao Lu, Zhiyong Fang, Siqi Wang, Zhenfei Zhang, Yongzheng Jia, Dawn Song, and Jiaheng Zhang. 2025. zkPyTorch: A Hierarchical Optimized Compiler for Zero-Knowledge Machine Learning. *Cryptology ePrint Archive*, Paper 2025/535.
- [31] Jianzhu Yao, Hongxu Su, Taobo Liao, Zerui Cheng, Huan Zhang, Xuechao Wang, and Pramod Viswanath. 2026. TAO: Tolerance-Aware Optimistic Verification for Floating-Point Neural Networks. In *Proceedings of the 21st European Conference on Computer Systems (EuroSys)*. ACM, 1515–1532. doi:10.1145/3767295.3803612
- [32] Dmytro Zakharov, Oleksandr Kurbatov, Artem Sdobnov, Lev Soukhanov, Yevhenii Sekhin, Vitalii Volovyk, Mykhailo Velykodnyi, Mark Cherepovskyi, Kyrilo Baibula, Lasha Antadze, Pavlo Kravchenko, Volodymyr Dubinin, and Yaroslav Panasenko. 2025. Bionetta: Efficient Client-Side Zero-Knowledge Machine Learning Proving. arXiv:2510.06784.
- [33] Zkonduit. 2026. EZKL: An Engine for Doing Inference for Deep Learning Models and Other Computational Graphs in a zk-SNARK. <https://github.com/zkonduit/ezkl>. GitHub repository, accessed September 2026.